MINI PROJECT – CRYPTOGRAPHY

Project Title: IoTShield – Secure IoT Device Authentication using HMAC-SHA256 Challenge–Response, with Signed Firmware and Shared Key Custody

Student Name(s): Gargi Deore, Diya Jairamani | **Roll No(s):** 25030421013, 25030421011 | **Class/Division:** B.Sc | **Faculty Guide:** Dr. Reema Roychoudhary | **Department:** SAII | **Academic Year:** 2026 – 27

1. PROBLEM STATEMENT

The problem. IoT devices (sensors, cameras, GPS trackers, smart locks) talk to servers over networks anyone can listen to. A server that only trusts a device ID, or a password sent over the network, can be fooled by **spoofing**, by a **replayed** recording of a valid login, or by one **leaked shared secret** that exposes every device. Attackers also push **malicious firmware** that devices install blindly, and the one **master key** that protects every device secret is a single file that can be stolen or lost.

Why it matters. Unattended, always-on devices are easy doors into homes, factories and vehicle fleets. One forged identity or poisoned update can compromise the whole network.

The need. Prove a device owns a secret **without ever sending the secret**, prove an update really comes from its maker, and make sure **no single person or file** holds the master key.

2. OBJECTIVES

1. Authenticate every device by **proof of key ownership**: the secret key is never transmitted.
2. Make recorded or forged responses useless with **single-use random challenges** that expire in 2 minutes.
3. **Protect the keys:** device keys encrypted at rest (Fernet); the master key can be **split among people** (Shamir), sealed off the disk and rotated; admin login needs a password + emailed code.
4. Accept firmware only with a valid **Ed25519 signature**, and record every attempt in an **audit log**.

3. CRYPTOGRAPHIC CONCEPT

Algorithm / Protocol:
Challenge–response with **HMAC-SHA256** (RFC 2104) · **Ed25519** signatures (RFC 8032) · **Fernet** authenticated encryption · **Shamir's Secret Sharing** (k-of-n)

Type:
Symmetric MAC (device login) · asymmetric digital signature (firmware) · symmetric AEAD, AES-128-CBC + HMAC (keys at rest) · secret sharing for the master key

Key / Parameters:
256-bit random device key · 256-bit random challenge, one use, 120 s expiry · 32-byte Ed25519 key pair · master key split 3-of-5 over a 521-bit prime field · 6-digit admin code

Security Properties:
Authentication · integrity · freshness (anti-replay) · constant-time check · confidentiality of stored keys · firmware origin · **k-1 shares reveal nothing**

4. REAL-WORLD APPLICATIONS

- **Secure Communication:** prove identity before a device may talk to a server
- **Data Protection:** keys encrypted at rest; master key split among custodians, as in key-management systems
- **Authentication:** smart locks, sensors, cameras, GPS trackers
- **Digital Signatures:** signed firmware / over-the-air updates
- **E-commerce / Banking:** HMAC-signed API calls, emailed one-time codes (2FA)
- **IoT / Cloud Security:** authenticate and audit a whole fleet
- **Others:** smart homes, industrial IoT, vehicle fleets

5. SYSTEM ARCHITECTURE / DESIGN

A · DEVICE LOGIN (CHALLENGE–RESPONSE)

IoT Device (Input) "I am TEMP-001" → Random Challenge 32 bytes, from secrets, valid once, 2 min → Secret Key 256-bit, kept in the device, never sent → HMAC-SHA256 (Algorithm) key + challenge → Server Verifies recompute + constant-time compare → GRANTED / DENIED (Output) logged to SQLite

B · SIGNED FIRMWARE UPDATE (PUBLIC-KEY SIGNATURE)

Admin Uploads firmware image → SHA-256 hash of the image → Ed25519 Sign private key stays on the server → Device Downloads image + signature → Verify + Anti-rollback built-in public key → Install / Refuse HMAC-signed report

C · KEY CUSTODY (SHAMIR SECRET SHARING)

Master Key 256-bit, encrypts every device key → Split k-of-n 3 of 5 shares, shown once, never stored → Custodians each person keeps one share → Seal key file deleted from the disk → Unseal 3 shares rebuild it, in memory only → Rotate new key, all device keys re-encrypted

Admin browser (HTML · CSS · JS) ⇄ **Flask REST API** (app.py) ⇄ **SQLite** (devices · challenges · logs · firmware) · crypto.py · shamir.py · custody.py hold the cryptography and key custody

6. METHODOLOGY / WORKFLOW

STEP 1 — Requirement Analysis
threats: spoofing, replay, key theft, bad firmware

STEP 2 — Keys & Custody
256-bit keys, Fernet vault, Ed25519 pair, Shamir shares

STEP 3 — Challenge–Response
random challenge → HMAC → constant-time verify

STEP 4 — Signed Firmware
SHA-256 + Ed25519, version signed, no rollback

STEP 5 — Testing & Verification
88 automated tests, attack simulator

Attacks simulated, all refused and logged:

- Wrong key → **denied**
- Replayed response → **blocked**
- Forged challenge → **rejected**
- Bit-flipped firmware → **refused**
- Attacker-signed firmware → **refused**
- Old firmware relabelled as new → **refused**
- Downgrade to older version → **refused**
- Stolen key file → **vault sealed**
- Leaked share → **key rotated, old shares dead**

7. TECHNOLOGY STACK

Programming Language
Python 3 · JavaScript (ES6)
HTML5 · CSS3 · SQL

Tools / Platform
Flask 3 web framework · SQLite
REST / JSON API · browser dashboard
always-on background service (launchd)

Libraries
`hmac · hashlib · secrets (standard library)`
cryptography (Fernet, Ed25519) · python-dotenv
Shamir's scheme: our own implementation

Hardware (if any)
MacBook Air · simulated IoT devices
(sensor, camera, motion, GPS) via device_simulator.py

IoTShield

MINI PROJECT – CRYPTOGRAPHY

IMPLEMENTATION | ANALYSIS | IMPACT

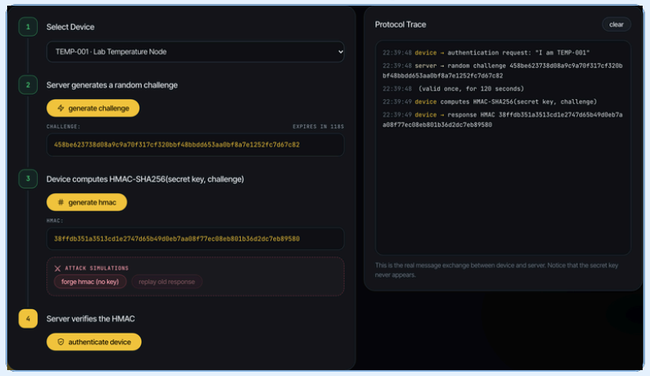
SAII(साई) SYMBIOSIS ARTIFICIAL INTELLIGENCE INSTITUTE

8. IMPLEMENTATION

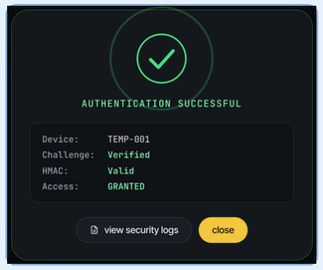
- **Approach:** Flask REST API, SQLite, a device simulator that acts like firmware over HTTP, and an admin console.
- **Modules:** crypto.py (all cryptography), shamir.py, custody.py, database.py, app.py (routes and protocol).
- **Key generation:** secrets.token_hex(32) gives a 256-bit key, stored Fernet-encrypted.
- **Authentication:** the device returns HMAC-SHA256 of a one-use challenge; the server recomputes it and compares in constant time.
- **Firmware:** Ed25519 signature over version and SHA-256; devices verify and refuse older versions.
- **Key custody:** master key split with Shamir (3 of 5), sealed off the disk, rotated if a share leaks.
- **Input / data:** 4 demo devices, 3 signed firmware releases, 88 automated tests.

```
# utils/crypto.py
def compute_hmac(secret_key_hex,
challenge_hex):
    key_bytes = bytes.fromhex(secret_key_hex)
    message = challenge_hex.encode("ascii")
    return hmac.new(key_bytes, message,
                    hashlib.sha256).hexdigest()
# server side
hmac.compare_digest(expected, received)
```

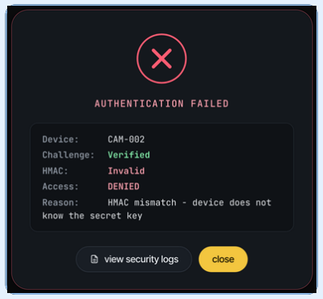
9. WORKING / OUTPUT (SCREENSHOTS)



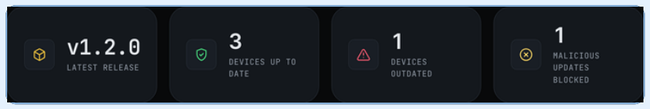
Screenshot 1 – Authentication. Challenge and HMAC are generated; the protocol trace shows the secret key never appears.



Screenshot 2 – genuine device: HMAC valid, **access granted**



Screenshot 3 – forged HMAC (no key): **access denied**



Screenshot 4 – Signed firmware. Latest release, devices up to date, and **1 malicious update blocked**.

10. RESULTS & ANALYSIS

88	0	6 / 6	3-of-5
automated tests passing	tests failed	attack types refused	shares rebuild the key

#	Test Case	What was tried	Result	Why
1	Genuine device login	Correct key	ACCESS GRANTED	HMAC matches
2	Forged HMAC	Wrong key	ACCESS DENIED	Needs the 256-bit key
3	Replay of a response	Same challenge again	Blocked	One use, 120 s expiry
4	Tampered firmware	One bit flipped	Refused	Hash changes; signature fails
5	Attacker-signed firmware	Attacker's own key	Refused	Only the server's key is trusted
6	Old image relabelled new	Version text changed	Refused	Version is in the signed text
7	Genuine older firmware	Older, validly signed	Refused	Version must be newer
8	Key rebuild, 2 of 3 shares	Too few shares	No match	Fewer than k reveal nothing

11. SECURITY ANALYSIS

- **Confidentiality:** device keys are Fernet-encrypted; a sealed vault leaves only encrypted keys on disk.
- **Integrity:** the signature covers version and SHA-256, so one changed bit fails.
- **Authentication:** devices prove key ownership; the admin needs a password and an emailed code.
- **Key management:** master key split 3-of-5, sealed, rotated if a share leaks.
- **Threats considered:** spoofing, replay, tampered firmware, downgrade, stolen key file, leaked share.
- **Limitations:** needs HTTPS; admin password not hashed; no HSM; devices simulated.

12. CONCLUSION

- **Key finding:** a server can verify a secret it never receives, using HMAC-SHA256 on a fresh challenge.
- **Objectives:** forged and replayed logins and bad firmware were refused, and no single person or file holds the master key.
- **Outcome:** a working web app with 88 passing tests.
- **Demonstrates:** how MACs, signatures, encryption and secret sharing work together for IoT trust.

13. FUTURE SCOPE

- TLS with mutual authentication.
- Secure element or HSM, and secure boot on the device.
- Per-device key rotation and revocation.
- Threshold signing for the firmware key.
- Anomaly detection and tamper-evident logs.
- A run on real hardware (ESP32 or Raspberry Pi).

14. REFERENCES

- W. Stallings, *Cryptography and Network Security*
- RFC 2104 (HMAC) and RFC 8032 (EdDSA)
- A. Shamir, *How to Share a Secret*, CACM 22(11), 1979
- NIST FIPS 198-1, FIPS 180-4, SP 800-57
- Python docs (hmac, hashlib, secrets), cryptography package, Flask